Ders 11 Genişletilmiş Notu: Seminer Sunumları: DNN ile Sismik Veri Sıkıştırma, Dil Modelleriyle Metin Sıkıştırma, NLP için Bayesçi Sıkıştırma, Evrişimli Otokodlayıcı ve Draco-ROS Geometri Sıkıştırma

Ders 11 Genişletilmiş Notu: Seminer Sunumları: DNN ile Sismik Veri Sıkıştırma, Dil Modelleriyle Metin Sıkıştırma, NLP için Bayesçi Sıkıştırma, Evrişimli Otokodlayıcı ve Draco-ROS Geometri Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: Derin Sinir Ağı ile Tahmin Tabanlı Sismik Veri Sıkıştırma
- 3. Sunum 2: Dil Modelleriyle Metin Sıkıştırma
- 4. Sunum 3: Doğal Dil İşleme için Bayesçi Sıkıştırma
- 5. Sunum 4: Evrişimli Otokodlayıcılarla Sismik Veri Sıkıştırma
- 6. Sunum 5: Draco ile ROS'ta 3B Örgü ve Nokta Bulutu Sıkıştırma
- Örnek Sorular

1. Dersin Yapısı

Bu dersle birlikte seminer haftaları başladı. Her sunumda bir öğrenci, veri sıkıştırmayla ilgili bir konferans ya da sempozyumda yayımlanmış bir makaleyi anlattı. Hoca, sunumlar sırasında anlamadığı yerleri sorduğunu, sunumdan sonra da makaleyi derste görülen kavramlara bağlayan sorular sorduğunu belirtti. Bu derste beş sunum yapıldı: iki tanesi sismik veri sıkıştırma, iki tanesi doğal dil işlemeyle ilgili sıkıştırma, biri de 3B geometri verisinin sıkıştırılmasıydı.

Dördüncü sunum İngilizce yapıldı ve altyazının bu bölümü (yaklaşık 01:29:00-01:39:00) çoğu yerde anlamsız tekrarlara düştü. Bu sunumun anlatımı büyük ölçüde ekrandaki slaytlardan ve hocanın sorularından kuruldu.

2. Sunum 1: Derin Sinir Ağı ile Tahmin Tabanlı Sismik Veri Sıkıştırma

İlk sunumda bir öğrenci, 2019 yılında bir konferansta sunulmuş "Seismic Data Compression Using DNN" konulu makaleyi anlattı.

2.1 Problem

Sismik veriler, karada ve denizde petrol ve gaz araması için farklı konumlara yerleştirilmiş bir dizi sensör tarafından toplanır. Bu veriler büyük depolama alanı ister. Ayrıca veri işleme merkezlerine iletilmeleri gerektiği için yüksek bant genişliğine ihtiyaç duyulur. Bu nedenle sismik verinin sıkıştırılması ciddi bir ihtiyaçtır.

Sunan öğrenciye göre sismik verilerin kayıpsız sıkıştırılması üzerine birçok çalışma vardır, ancak kayıpsız yöntemlerle iyi bir sıkıştırma oranı elde edilememiştir. Entropi ve tahmin tabanlı kayıpsız yaklaşımlar daha iyi sonuç vermiştir. Son dönemde bunlara alternatif olarak **kayıpsıza yakın kayıplı** sıkıştırma ön plana çıkmıştır; makale de bunun bir örneğidir. Hoca burada "kayıplının mı örneğini göreceğiz?" diye sordu; cevap, kayıpsıza yakın kayıplı bir yöntem olduğuydu.

2.2 Temel terimler

- **Tahmin tabanlı öğrenme:** $n+1$. örnek, kendinden önceki n örneğin değerleri kullanılarak tahmin edilir. Tahmin edilen örnek önceki örneklerle ilişkili olarak bulunur.
- **Residual (fark) değer:** orijinal veri ile tahmin edilen veri arasındaki farktır. Sıkıştırmanın şartı, fark değerlerinden geri dönülerek orijinal verinin bulunabilmesidir.
- **Nicemleme (quantization):** fark değerlerinin ayrıklaştırılması; ardından bilgisayarın anlayacağı ikili biçime çevrilmeleri.

2.3 Tahmin tabanlı sıkıştırmanın iki aşaması

Tahmin tabanlı sıkıştırma iki aşamadan oluşur. İlk aşamada sismik verinin bir kısmı, özellikle ilk n değer, tahmin modelini eğitmek için kullanılır. Ardından orijinal ve tahmin arasındaki fark değerleri hesaplanır. Bu fark değerleri Huffman gibi bir algoritmayla sıkıştırılır. Tahminde kullanılan parametreler sıkıştırılmış verinin başlığına yerleştirilir ve veriyle birlikte iletilecek yere gönderilir.

Alıcı taraf, başlıktaki parametreleri kullanarak aynı tahmin değerlerini hesaplar. Huffman ile sıkıştırılmış veriyi açarak fark değerlerini yeniden bulur. Tahmin ve fark değerlerini birleştirerek orijinal değerleri elde eder. Tahmin için LPC (Linear Predictive Compression) ya da derin sinir ağı (DNN) gibi yöntemler kullanılır.

2.4 LPC ve DNN

LPC, otoregresif bir model kullanarak ayrık zamanlı serilerde kendinden önceki değerlerden doğrusal korelasyona dayalı bir tahmin yapar. Daha iyi sonuç için kayıpsız sıkıştırmada tercih edilir, ancak verinin karmaşıklığına göre sıkıştırma oranı düşük kalabilir. Tahmindeki ilişki doğrusal olduğu için kayıplı sıkıştırmada geri çözümlemede hata payı fazla olur; hatanın etkisini azaltmak için normalizasyon kullanılabilir.

DNN modeli bir giriş serisinden ve birden çok gizli katmandan oluşur. Tahmin LPC'ye benzer şekilde önceki değerlerden yapılır, ancak burada doğrusal değil, doğrusal olmayan bir ilişki kurulur. DNN'de tahmin daha hassas olabildiği için fark değerlerinin varyansı daha küçük çıkar.

2.5 Restricted Boltzmann Machine (RBM)

Makalede özel olarak ele alınan RBM, iki katmanlı bir yapıdır. İki katman arasındaki düğümler birbiriyle ilişkilendirilir; aynı katmandaki düğümler arasında ilişki yoktur. "Restricted" (kısıtlı) adı bu kısıttan gelir (ekrandan, 00:22:30). Her kenarın bir ağırlığı vardır. Görünür (visible) katmandan gelen girdi değerleri ağırlıklarla çarpılır, gizli katmanın bias değeriyle toplanır ve bir sigmoid fonksiyonundan geçirilerek çıkış elde edilir. Ağırlıklar eğitim sırasında ileri ve geri geçiş algoritmalarıyla güncellenir.

Çalışmada çok sayıda gizli katmanı olan DNN için optimuma yakın başlangıç ağırlıkları hesaplamak amacıyla RBM kullanıldı; böylece ağırlıklar daha gerçekçi değerlerle başlar.

2.6 Çalışmanın aşamaları

Slayttaki aşamalar şunlardır (ekrandan, 00:20:00):

- DNN ve RBM (Restricted Boltzmann Machine) ile veri eğitimi
- Supervised (denetimli) yöntemler kullanılarak tahmin değerlerinde ince ayar
- Fark (residual) değerlerin hesaplanması ve nicemlenmesi (quantization)
- Değişken kod uzunluğu için Huffman kodlama

2.7 Sonuçlar

Veri seti olarak bir Texas veri seti kullanıldı ve veri eğitim, doğrulama ve test olmak üzere üç kısma ayrıldı. Aynı veriler hem DNN hem de LPC'de kullanılıp sonuçlar karşılaştırıldı. Fark değerlerinin dağılımını gösteren grafikte DNN'in fark değerlerinin varyansı sıfıra yakın çıktı; LPC'nin farkları daha geniş, orijinal veri ise daha homojen dağılmıştı. Bilgi teorisi açısından düşük varyans, daha düşük entropi ve daha yüksek sıkıştırma oranı demektir.

Nicemleme seviyelerine göre fark değerleri için üç kod kitapçığı oluşturuldu. SNR (Signal-to-Noise Ratio) sonuçları şöyledir; L_Q nicemleme seviyesini, N veri segmenti değerini gösterir (ekrandan, 00:21:50):

	LPC			DNN		
N	L_Q=16	LPC 32	LPC 64	L_Q=16	DNN 32	DNN 64
5	23,1	28,96	30,27	43,02	52,15	58,31
10	26,56	27,64	29,55	42,07	51,45	57,41
15	25,86	28,69	29,88	42,81	52,81	58,82

DNN bütün konfigürasyonlarda LPC'den daha yüksek SNR verdi (değerler dB).

Fark değerlerinin sıkıştırılmasında iki kodlama karşılaştırıldı: sabit uzunluklu uniform kodlama ve değişken uzunluklu Huffman kodlama. Sıkıştırma oranları (CR) şöyledir (ekrandan, 00:22:30):

	L_Q=16,			L_Q=32,			L_Q=64,		
CR	n=5	10	15	n=5	10	15	n=5	10	15
Uniform	7,8	7,6	7,4	6,3	6,2	6,07	5,3	5,2	5,1
Huffman	14,4	13,3	12,2	11,04	10,7	9,6	8,75	8,7	7,9

Her durumda Huffman daha yüksek sıkıştırma oranı verdi. Nicemleme seviyesi arttıkça SNR yükselir, sıkıştırma oranı düşer. Orijinal sinyal ile DNN ile kodlanıp çözülmüş sinyal üst üste konduğunda aradaki farkın çok küçük olduğu, kayıplı olmasına rağmen neredeyse kayıpsız gibi bir sonuç elde edildiği gösterildi. Sunumda yaklaşık 10:1 sıkıştırmada 40 dB'den yüksek SNR elde edildiği söylendi.

2.8 Hocanın sorusu: blok blok sıkıştırma

Hoca bu çalışmayı derste görülen bir örneğe bağladı. İkinci ya da üçüncü haftada 7-8 farklı sembolü olan bir veride farklar alınarak sembol sayısının 1 ve -1 gibi iki sembole indirildiğini hatırlattı. Burada da benzer bir residual sıkıştırma yapılıyor.

Ardından şunu sordu: bir deprem verisi düşünülsün. Önce hafif salınımlar vardır, depremden önce belki büyük salınımlar olur, sonra işaret yeniden sönümlenir. Böyle bir işarette farklar alındığında büyük salınım kısmında varyans büyür. Bu durumda verinin tamamı tek blok olarak mı, yoksa küçük bloklara bölünerek mi sıkıştırılıyor? Sunan öğrenci verinin bölünerek sıkıştırıldığını, $N = 5, 10, 15$ segment değerleriyle çalışıldığını söyledi. Hoca bunun böyle olması gerektiğini, aksi halde varyansın aralarda büyüyeceğini vurguladı.

> **Hocanın vurgusu:** Fark alınarak sıkıştırılan bir işarette büyük salınımlı bölgeler farkların varyansını büyütür. Bu yüzden veri küçük bloklara bölünerek sıkıştırılmalıdır.

3. Sunum 2: Dil Modelleriyle Metin Sıkıştırma

İkinci sunumda bir öğrenci, dil modellerine dayalı bir metin sıkıştırma makalesini anlattı. Sunum doğal dil işleme ve veri sıkıştırma, entropi ve Huffman kodlama, dil modelleri, sözlük hazırlama ve kategorizasyon, kayan bağlam penceresi ve son olarak performans ile perplexity ilişkisi başlıklarından oluştu.

3.1 Fikir

Makale, dil modellerine dayalı bir sıkıştırma algoritmasının nasıl yapılabileceğini ve elde edilen olasılık tahminlerinin test dosyaları için kod kelimeleri oluşturmakta nasıl kullanılabileceğini anlatır (ekrandan, 00:30:00). Geleneksel sıkıştırma algoritmaları kelimelerin birbirini izleme olasılığını belirleyemez; yalnız daha önce karşılaştıkları kelime ve karakterlerden bir olasılık çıkarır. Makale bu eksikliği dil modelleriyle gidermeyi amaçlar.

Bir belge birden fazla konu içeriyorsa kelimelerin olasılıkları konudan konuya değişebilir. Bu yüzden yöntem, belgenin farklı bölümlerinin ayrı ayrı sıkıştırılıp sonra birleştirilmesini sağlayabilir. Benzer şekilde belge içindeki farklı dil türleri tanımlanıp her biri için ayrı bir sözlük kullanılarak daha iyi bir sıkıştırma elde edilebilir.

3.2 Entropi ve Huffman

Entropi, tekrar eden karakter, kelime ya da örüntülerin daha kısa sembollerle temsil edilmesinin temelidir. Sıkıştırma yöntemleri genellikle sıkıştırılacak verinin kendisinden kelime dağılımını tahmin eder. Ancak belge kısaysa yeterli veri olmaz ve tahmin edilen dağılım gerçekçi dağılımdan uzaklaşabilir. Huffman kodlama entropi kavramına dayanır ve sık kullanılan kelimelere daha kısa kodlar atar. Dil modeli ise bir kelimenin önceki kelimelere göre ne kadar olası olduğunu tahmin ederek konuya daha özgü bir sözlük oluşturulmasını sağlar.

3.3 Dil modelinin sıkıştırmadaki rolü

Sunan öğrenciye göre karakter sıkıştırmada dil modellerinin getirdiği fark küçüktür; ASCII tabanlı ilerlendiğinde benzer sonuçlar elde edilir. Dil modelleri asıl farkı kelime sıkıştırmada yaratır: bir belgede hangi kalıpların daha sık kullanılacağını geçmiş bilgisine dayanarak tahmin edebilir.

Slayttaki İngilizce trigram örneğinde “will have to” ile “will have it” kalıpları karşılaştırıldı. Bir insan “will have to” kalıbının daha sık geçeceğini tahmin eder, çünkü bu kalıp herhangi bir konuya bağlanabilir. “Will have it” ise daha dar bir kapsamda kullanılır. Dil modeli bu insan gibi tahmin etme becerisini sıkıştırmaya taşır.

3.4 Algoritma

Slayttaki algoritma şöyledir (ekrandan, 00:34:40):

1. Bir dil modeli eğitilir. Bu model unigram, bigram ve trigram olasılıklarını hesaplar.
2. Her bigram için üçüncü kelimenin gözlemlenen herhangi bir kelime ya da UNK olma olasılığı hesaplanır.
3. Her bigram için üçüncü kelimenin dağılımına dayalı bir Huffman ağacı oluşturulur.
4. Kelimelerin olasılıklarına dayalı bir unigram Huffman ağacı oluşturulur.
5. Kelime kodlaması yapılır: kelimeler ilgili Huffman ağacındaki karşılık gelen bitlerle değiştirilir.

UNK simgesi bilinmeyen, daha önce karşılaşılmamış kelimeleri temsil eder. Bilinmeyen kelimeler UNK ile kodlanır ve daha düşük bir sıkıştırma modeline geçilir. Unigram modeli bile kelimeyi tanımlayamazsa karakter sıkıştırmaya (kelimenin ASCII karşılığıyla) geçilir.

3.5 Sözlük hazırlama, kategorizasyon ve kayan bağlam penceresi

Sıkıştırılacak belge birden fazla konu içerebilir; örneğin günlük hayattan bir girişle başlayıp teknik ayrıntıya geçen bir belge. Bir pencere, içindeki kelimeleri gözlemleyerek belgenin hangi konuda olduğunu tahmin eder ve doğru sözlük kullanılır. Belgenin diline göre özelleştirilmiş sözlükler de kullanılabilir. Örneğin İngilizce bir metinde Almanca kelimeler geçiyorsa, sıradan bir yöntem her Almanca kelimeyi tanımadığı için daha uzun kodlarla kodlar; dile özgü sözlük ise bunu önler.

Soru: Konunun değiştiği nasıl anlaşılıyor? Konu değişince olasılıklar yeniden mi belirleniyor?
Cevap: Sunan öğrenciye göre olasılıklar yeniden hesaplanmaz; her konu için önceden çıkarılmış bir Huffman ağacı vardır. Pencerenin belirli bir kelime sınırı vardır ve tek bir kelimeye

değil pencere içindeki kelimelere bakılarak bir konu ataması yapılır. Hoca bunu şöyle özetledi: kayan pencereye yeni bir kelime geldiğinde, bu kelime pencere içindeki hiçbir kelimeyle örtüşmüyorsa (LZ yöntemlerinde anlatıldığı gibi) bu kelime daha önce hiç geçmemiş demektir; demek ki konu, yani bağlam değişmiştir.

Slayta göre kayan bağlam penceresinin faydaları şunlardır (ekrandan, 00:40:00):

- Sözcüklerin birbirleriyle olan ilişkilerini ve olası kombinasyonlarını dikkate alarak sıkıştırma yapar.
- Her sıkıştırılmış sözcük için bir önceki ve bir sonraki sözcük dahil birkaç sözcüğü içerir. Bu bağlam penceresi modelin öğrenme kabiliyetini artırır ve daha kesin bir kodlama sağlar.
- Belgenin kategorize edilmesi gereken durumlarda kullanılır. Belge birden fazla kategoride bilgi içeriyorsa her kategori için ayrı bir sözlük oluşturulur ve ayrı kodlama yapılır.

3.6 Perplexity ve sonuçlar

Perplexity, bir dil modelinin ne kadar iyi tahmin yaptığını ölçer ve entropiyle ilişkilidir. Entropide olduğu gibi belge içindeki verilerin olasılık tahminlerine dayanır. Düşük perplexity, modelin daha doğru tahmin yaptığını gösterir. İyi bir perplexity veren dil modeli bulmak, iyi sıkıştırmaya izin veren düşük entropili bir dağılım bulmakla eşdeğerdir.

> **Ek bilgi (derste anlatılmadı):** Perplexity genellikle $PP = 2^H$ olarak tanımlanır; burada H, modelin kelime başına bit cinsinden çapraz entropisidir. Bu yüzden perplexity'yi düşürmek, kelime başına gereken ortalama bit sayısını düşürmek anlamına gelir.

Deneylerde eğitim verisindeki trigram sayısı arttıkça perplexity değerinin düştüğü görüldü. Bu, daha önce karşılaşılan trigramların daha olası hale geldiğini ve sıkıştırmanın daha etkili olduğunu gösterir (ekrandan, 00:41:57). Sunumda 0,36 sıkıştırma oranıyla gzip ve bzip2 gibi bilinen algoritmalarla rekabet edebilecek bir seviyeye ulaşıldığı söylendi.

3.7 Soru-cevap

Soru (hoca): Sıkıştırma oranı 0,36 verildi. Bunu sıkıştırma performansı olarak verseydik %64 sıkıştırma mı olurdu? **Cevap:** Evet. Sıkıştırılmış boyut orijinalin 0,36'sı ise kazanç $1 - 0{,}36 = 0{,}64$, yani %64'tür.

Soru: Sözlükte kelimelerin tekli, ikili ve üçlü eşleşmeleri kullanılıyor. Bu, Huffman ağacını çok büyütmez mi? **Cevap:** Sunan öğrenci, makalede entropi değerinin verilmediğini, yalnız gzip, bzip2 gibi araçlarla sıkıştırma oranı karşılaştırması yapıldığını söyledi. Hoca, entropi verilmiş olsaydı ortalama kaç bit kullanıldığından ağacın derinliği hakkında bir fikir edinilebileceğini belirtti. Önce üçlü, yoksa ikili, yoksa tekli, o da yoksa karakter düzeyinde arama yapılıyorsa ve hepsi aynı ağaçtaysa ağacın seviyesi yüksek olacaktır. Belki her bağlam değiştiğinde ağaç yeniden oluşturuluyordur ya da belge paragraf paragraf işleniyordur; bu durumda ağaç çok derin olmaz. Pencerenin boyutu da önemlidir. Hoca ayrıca gzip'in de Huffman kullandığını, dil modeli denilen n-gram'ların karakter tabanlı da, kelime tabanlı da olabileceğini ekledi.

4. Sunum 3: Doğal Dil İşleme için Bayesçi Sıkıştırma

Üçüncü sunumda bir öğrenci, Moskova'daki araştırmacılar tarafından yazılmış "Bayesian Compression for Natural Language Processing" makalesini anlattı (ekrandan, 00:50:00). Sunan öğrenci, makalenin derste görülen klasik sıkıştırma mantığından farklı olarak yapay zeka yaklaşımıyla yazıldığını söyledi. Hoca da başlıkta "Bayes" geçiyorsa mutlaka ağır bir matematik olduğunu, o kısmın kapalı kutu gibi düşünülerek anlatılabileceğini belirtti.

4.1 Doğal dil işleme ve RNN

Doğal dil işleme, bilgisayarların insan dilini kavramasını, üretmesini ve işlemesini sağlayan bir yapay zeka alanıdır. Metin sınıflandırma, makine çevirisi, ses tanıma, yazım denetimi, soru cevaplama gibi işlerde kullanılır.

Tekrarlayan sinir ağları (RNN), gizli durumlarıyla önceki çıktıların girdi olarak kullanılmasına izin veren bir sinir ağı sınıfıdır. Avantajları: herhangi bir uzunluktaki girdiyi işleyebilir, model boyutu girdi büyüklüğüyle artmaz, geçmiş bilgiyi hesaba katar (bir hafızası vardır), ağırlıklar zaman içinde paylaşılır. Dezavantajları: boyut arttıkça hesaplama yavaşlayabilir, uzun zaman önceki bilgiye erişmek zordur (sunan öğrenciye göre vanishing gradient denen olay) ve mevcut durum için gelecekteki girdiler hesaba katılamaz.

4.2 Motivasyon ve katkı

Makalenin motivasyonu, doğal dil işlemede yaygın kullanılan RNN'lerin çok fazla parametreye sahip olması ve sınırlı belleği olan taşınabilir cihazlarda kullanılabilmeleri için sıkıştırılmaları gerekmesidir. Makine çevirisi gibi karmaşık görevler için modern RNN mimarileri çok sayıda parametre içerir.

Mutlak değeri küçük ağırlıkları modelden çıkaran budama (pruning) yöntemleri sezgiseldir ve zaman alıcı hiperparametre ayarı ister. Yazarlar bunu genelleştirilebilir bir matematiksel modele oturtur. **Sparse VD** (seyrek varyasyonel dropout) adı verilen modelde parametreler eğitim sırasında kendiliğinden ayarlanır. NLP görevlerinde RNN ağırlıklarının çoğu kelime dağarcığına bağlı olarak gömme (embedding) katmanında yoğunlaşır. Makaleye göre bazı görevlerde doğru tahmin için kelimelerin çoğu gereksizdir. Bu gereksiz kelimeleri budamak için **çarpımsal ağırlıklar** eklenir. Eğitim sırasında sıfırlanan çarpımsal ağırlıklar gereksiz kelimelerin filtrelenmesini sağlar ve RNN'in seyreklik düzeyini artırır.

4.3 İlgili çalışmalar

RNN boyutunu azaltmaya yönelik önceki çalışmalar üç sınıfa ayrılır:

- Ağırlık matrislerinin tahmini
- Ağırlık matrislerinin hassasiyetinin azaltılması
- Ağırlık matrislerinin seyreltilmesi (pruning)

Bu çalışmanın yöntemi seyreltmedir. Budama, ağırlıkların bir eşik değere göre kesilmesidir. Literatürde bir çalışmada eşik değeri, ağırlıkların çıkarılma sıklığını, oranını ve süresini kontrol eden birkaç hiperparametreyle seçilmiştir. Başka bir çalışma LSTM'deki ağırlıkları her nörona karşılık gelen gruplar halinde budamayı önermiştir. Bayesçi sinir ağlarına dayanan seyreltme yöntemi daha az parametreye ve daha yüksek sıkıştırma performansına sahip olduğu için tercih edilmiştir. Bayesçi sinir ağlarında ağırlıklar rastgele değişkenler olarak ele alınır ve seyrek ağırlık isteği bir önsel (prior) dağılımla ifade edilir. Eğitimde önsel dağılım sonsal (posterior) dağılıma dönüştürülür; test aşamasında tahmin için bu kullanılır. Bayesçi seyreltmenin temel avantajları budama tabanlı yöntemlere göre daha az hiperparametreye sahip olması, bu yüzden hızlı olması ve daha yüksek seyreklik düzeyine izin vermesidir.

4.4 Yöntem

Çalışmada önyargılar (bias) ayıklanmış ve seyreltmeye dahil edilmemiştir. Standart RNN mimarisi şöyledir (ekrandan, 00:57:05):

$$y = [x_1, \dots, x_T]$$

$$\text{embedding: } \tilde{x}_t = w^e_{x_t}$$

$$\text{recurrent: } h_{t+1} = \sigma(W^h h_t + W^x \tilde{x}_{t+1} + b^r)$$

$$\text{fully-connected: } \hat{y}_t = \text{softmax}(W^d h_t + b^d)$$

$$\omega = \{W^e, W^x, W^h, W^d\}, \quad B = \{b^r, b^d\}$$

Burada x girdi dizisi, y gerçek çıktı, $\widehat{y}$ RNN'in tahmin ettiği çıktıdır; y ve $\widehat{y}$ tek vektör ya da dizi olabilir. (X, Y) eğitim kümesini gösterir. Sapmalar hariç RNN'in bütün ağırlıkları ω ile, tek bir ağırlık matrisinin bir elemanı w_{ij} ile gösterilir.

Bayesçi kısım şöyledir (ekrandan, 01:00:00). Ağırlıkların önüne tamamen faktörize edilmiş log-uniform bir önsel konur:

$$p(\omega) = \prod_{w_{ij} \in \omega} p(w_{ij}), \qquad p(w_{ij}) \propto \frac{1}{|w_{ij}|}$$

Sonsal dağılım, tamamen faktörize edilmiş normal dağılımlı bir yaklaşımla temsil edilir:

$$q(\omega \mid \theta, \sigma) = \prod_{w_{ij} \in \omega} \mathcal{N}(w_{ij} \mid \theta_{ij}, \sigma_{ij}^2)$$

Sonsal yaklaşımın görevi varyasyonel alt sınır optimizasyonuna eşdeğerdir:

$$-\sum_{i=1}^{N} \int q(\omega \mid \theta, \sigma) \log p(y^i \mid x_0^i, \dots, x_T^i, \omega, B)\, d\omega + \sum_{w_{ij} \in \omega} KL(q(w_{ij} \mid \theta_{ij}, \sigma_{ij}) \,\|\, p(w_{ij})) \to \min_{\theta, \sigma, B}$$

$$KL(q(w_{ij} \mid \theta_{ij}, \sigma_{ij}) \,\|\, p(w_{ij})) \approx k \left(\frac{\sigma_{ij}^2}{\theta_{ij}^2} \right)$$

Hoca, formüllerdeki KL teriminin ve çift çizginin ne anlama geldiğini sordu, ancak ardından bu matematiğin ayrıntısına girmeye gerek olmadığını, RNN kullananların bile RNN formüllerinin nasıl türetildiğini çoğu zaman bilmediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** $KL(q \,\|\, p)$, Kullback-Leibler ıraksamasıdır ve iki olasılık dağılımının birbirinden ne kadar farklı olduğunu ölçer. Çift çizgi ($\|$) yalnızca iki dağılımı ayıran bir gösterimdir.

Yöntemin sözel özeti şöyledir: önce bütün ağırlıklar örneklenir, RNN uygulanır ve gradyan hesaplanır. Test sürecinde ortalama ağırlıklar kullanılır. θ bileşenlerinin çoğu sıfıra yaklaşır ve ağırlıklar seyrekleşir. θ bileşeni sıfıra yaklaştırılabiliyorsa o ağırlığın gereksiz olduğuna karar verilir ve ilgili kelime kelime dağarcığından çıkarılabilir. Bunun için sinyal-gürültü oranı düşük olan ağırlıkların bir eşiğin altında kalınca kaldırıldığı bir ölçüt kullanılır.

Soru (hoca): Yani RNN'deki parametre sayısını azaltabilmek için Bayes'ten yararlanıyoruz, değil mi? **Cevap:** Evet.

4.5 Deneyler

Yöntem iki alanda test edildi: metin sınıflandırma ve dil modelleme. Deneyler LSTM mimarisi üzerinde yapıldı ve üç model karşılaştırıldı: düzenleme olmayan temel model, Sparse VD ve kelime dağarcığı seyreltmesi için çarpımsal ağırlıklar eklenmiş Sparse VD-Voc.

Metin sınıflandırmada (ekrandan, 01:03:00):

- İkili sınıflandırma için IMDb, dört sınıflı sınıflandırma için AGNews veri kümesi kullanıldı.
- Eğitim verilerinin sırasıyla %15'i ve %5'i doğrulama için ayrıldı; her iki veri kümesi için en sık kullanılan 20.000 kelime alındı.
- Tek gömme katmanı 300 birimlik, gizli birim sayısı IMDb için 128, AGNews için 512 olan ağlar kullanıldı; LSTM'in son çıktısına tamamen bağlı bir katman uygulandı.
- Gömme katmanı word2vec ile başlatıldı. Sparse VD ve Sparse VD-Voc modelleri IMDb için 800, AGNews için 150 epok eğitildi.

Dil modellemede Penn Treebank veri kümesi üzerinde karakter düzeyi ve kelime düzeyi modelleme yapıldı. Kelime dağarcığı 50 karakter ya da 10.000 kelimeydi. Karakter düzeyinde 1000, kelime düzeyinde 256 gizli birimli bir LSTM katmanı ve bir sonraki karakteri ya da kelimeyi tahmin etmek için softmax katmanı kullanıldı.

Sonuç tabloları şöyledir; sıkıştırma $|w|/|w \neq 0|$, yani toplam ağırlık sayısının sıfır olmayan ağırlık sayısına oranıdır (ekrandan, 01:05:30):

Görev	Yöntem	Doğruluk %	Sıkıştırma	Sözlük	Nöron $\tilde{x}$ - h
IMDb	Original	84,1	1x	20000	300 - 128
IMDb	SparseVD	85,1	1135x	4611	16 - 17
IMDb	SparseVD-Voc	83,6	12985x	292	1 - 8
AGNews	Original	90,6	1x	20000	300 - 512
AGNews	SparseVD	88,8	322x	5727	179 - 56
AGNews	SparseVD-Voc	89,2	469x	2444	127 - 32

Görev	Yöntem	Valid	Test	Sıkıştırma	Sözlük	Nöron h
Char PTB (bits-per-char)	Original	1,498	1,454	1x	50	1000
Char PTB	SparseVD	1,472	1,429	4,2x	50	431
Char PTB	SparseVD-Voc	1,4584	1,4165	3,53x	48	510
Word PTB (perplexity)	Original	135,6	129,5	1x	10000	256
Word PTB	SparseVD	115,0	109,0	14,0x	9985	153
Word PTB	SparseVD-Voc	126,3	120,6	11,1x	4353	207

Metin sınıflandırmada doğruluk neredeyse hiç kaybolmadan çok yüksek sıkıştırma oranlarına ulaşıldı. Dil modellemede bu kadar yüksek oranlara ulaşılamadı, ancak Sparse VD'nin düzenleyici etkisi sayesinde temel modelden daha iyi kaliteye ulaşılarak modeller birkaç kat küçültüldü.

4.6 Tartışma: kayıplı mı, kayıpsız mı?

Sunan öğrenci, kendi yorumuna göre yöntemin derin öğrenmeyle kayıplı bir sıkıştırma yaptığını, ancak matematiksel bir modele oturduğu için sapmanın çok düşük olduğunu söyledi.

Soru (hoca): Makalenin girişinde ya da özetinde bunun kayıplı ya da kayıpsız sıkıştırma olarak kabul edildiğine dair bir bilgi var mıydı? **Cevap:** Hayır; bu sunan öğrencinin yorumuydu. Hoca, veri kümelerinin (haberler, film yorumları) metin olduğunu, metne kayıplı sıkıştırma uygulanamayacağını belirtti. Hocanın yorumuna göre makalede sıkıştırılan şey RNN'in kendisidir: RNN'in arkasındaki matematik Bayes kullanılarak basitleştirilmiş ve daha az parametreyle aynı başarı elde edilmiştir. Sunan öğrenci de modelin çok daha hızlı çalıştığını ve az yer kapladığını, ancak dil modellemede metin sınıflandırmadaki kadar başarılı olmadığını ekledi.

Bir başka öğrenci, sinyal-gürültü oranına göre yapılan elemenin, Bayesçi hesapların içinde özellik sayısını düşürerek daha doğru sonuç çıkarmaya yönelik olabileceğini yorumladı. Hoca, SNR'nin daha önce görüntü dosyalarında kullanıldığını hatırlattı: orijinal görüntü ile sıkıştırılıp açılmış görüntü arasındaki bozulmayı ölçmek için hesaplanıyordu. Burada ise sinyal-gürültü oranı düşük ağırlıklar ortadan kaldırılıyor; böylece katmanlar arasındaki bazı bağlantılar kullanılmaz hale gelir ve hesap hızlanır.

5. Sunum 4: Evrişimli Otokodlayıcılarla Sismik Veri Sıkıştırma

Aradan sonra dördüncü sunumda bir öğrenci, İngilizce olarak "Seismic Data Compression Using Deep Learning" makalesini anlattı. Bu bölümün altyazısı büyük ölçüde bozuk olduğu için anlatım slaytlardan ve okunabilen İngilizce cümlelerden kuruldu.

5.1 Sismik veri

Sismik veri, ses dalgalarını kullanarak yer altını değerlendirmek için kullanılan jeofizik veridir ve en çok petrol ve gaz endüstrisinde kullanılır (ekrandan, 01:30:00). Slayttaki şemada bir vibratör kamyonu yere titreşim gönderir, yansıyan dalgalar yüzeydeki jeofon gruplarıyla kayıt kanallarına alınır ve bir kayıt kamyonunda toplanır.

5.2 Modeller

- **Model 1: Evrişimli otokodlayıcı.** Bir kodlayıcı (encoder) ve bir kod çözücüden (decoder) oluşur. Kodlayıcı evrişim ve havuzlama (pooling) katmanlarıyla önemli özellikleri çıkarır ve veriyi sıkıştırır. Kod çözücü, kodlanmış katmandan çıkarılan özellikleri kullanarak evrişim ve katmanlarıyla veriyi yeniden kurar. Sonunda orijinal veriye yakın bir çıktı elde edilir.
- **Model 2:** Evrişimli sinir ağına dayanır ve birinci modelden farklı bir mimariye sahiptir. Evrişim katmanı, düzleştirme (flatten) katmanı ve yoğun (dense) katman olmak üzere bir dizi aşamadan oluşur.

5.3 Değerlendirme ölçütleri ve sonuçlar

Yeniden kurulan sinyalin kalitesini ölçmek için normalize ortalama kare hata (NMSE), normalize karekök ortalama kare hata (NRMSE) ve SNR kullanıldı.

Model 1'in 475 eğitim ve 119 test izi (trace) için farklı sıkıştırma oranlarındaki performansının bir kısmı şöyledir (ekrandan, 01:40:20):

CR	Eğitim süresi	NMSE (10^{-3})	NRMSE	SNR (dB)
100:1	1 dk 16 s	9,343	0,048	20,295
60:1	1 dk 4 s	8,181	0,045	20,872
50:1	57 s	7,597	0,044	21,194
30:1	1 dk 14 s	7,228	0,043	21,409
20:1	1 dk 7 s	6,460	0,040	21,898
15:1	1 dk 38 s	4,417	0,033	23,549
10:1	52,7 s	3,362	0,029	24,734
5:1	41,4 s	1,199	0,017	29,211
3:1	46,9 s	0,472	0,011	33,258
2:1	59,9 s	0,086	0,005	40,666

Sıkıştırma oranı arttıkça SNR düşer, NMSE ve NRMSE artar. Model 1 için en iyi SNR beş katmanla, Model 2 için ise iki katmanla elde edildi (CR = 30 için, ekrandan, 01:47:20). Yeniden kurulan sismik izler ELM ve DWT yöntemleriyle karşılaştırıldı; Model 2 orta ve yüksek sıkıştırma oranlarında en yüksek SNR'yi verdi. Sonuç slaytına göre iki model de sağlamdır ve iyi genelleme yeteneğine sahiptir; gelecekte farklı sismik bölgelerden daha büyük veri setleriyle test edilmeleri planlanmaktadır (ekrandan, 01:40:00).

5.4 Hocanın soruları

Soru (hoca): Bir tabloda sıkıştırma oranları 1:10, 1:100 gibi verildi; sonraki slaytlarda yalnız 10, 20, 100 gibi değerler var. Aralarında fark var mı? **Cevap:** Hayır, ikisi de aynı anlamdadır. Hoca 1:100'ü, 100 bitin 1 bite sıkıştırılması, 1:5'i ise 5 bitin 1 bite sıkıştırılması olarak açıkladı.

Ardından hoca tabloyu RMS hata ile SNR arasındaki ilişki üzerinden yorumladı. Derste RMS hata (Error Root Mean Square) ile SNR arasında ters orantı olduğu, birinin büyük olduğu yerde diğerinin küçük olması gerektiği anlatılmıştı. RMS hata, orijinal veri ile sıkıştırılıp açılmış veri arasındaki farktır; küçük olduğunda açılan veri orijinale daha yakındır. Tabloda da NRMSE küçükken SNR büyüktür.

Soru (bir öğrenci): SNR'ye bakınca sıkıştırma oranı arttıkça gürültü etkisinin arttığını ve bir bozulma olduğunu anlıyorum. Yanlış mı düşünüyorum? **Cevap:** Hoca, sıkıştırma büyüdükçe

bozulmanın arttığını; CR'ye, SNR'ye ya da ortalama kare hataya bakmanın aynı eğilimi farklı açılardan göstermek olduğunu söyledi.

> **Hocanın vurgusu:** RMS hata ile SNR ters orantılıdır. Biri büyükse diğeri küçük çıkmalıdır.

Hoca ayrıca birinci sunumdaki DNN çalışmasıyla karşılaştırma yaptı. Birinci sunumda 40 dB'nin üzerinde SNR değerleri elde edilmişti; bu tabloda ise 2:1'de yaklaşık 40 dB, 10:1'de yaklaşık 24 dB'dir. Buradan SNR'nin modelin mimarisine göre de değiştiği sonucunu çıkardı.

Soru (bir öğrenci): Model 1'in grafiğinde altı katman kullanıldığında özellik haritası sayısı 8'i geçince SNR çok keskin biçimde düşüyor. Bunun nedeni ne olabilir? **Cevap:** Sunan öğrenci makalede bunun nedeninin açıklanmadığını söyledi. Hoca da kesin bir yorum yapamadı; Model 2'de SNR'nin katmanlarla yavaş yavaş arttığını, iki grafiğin ölçeklerinin farklı olduğunu (birincisi yaklaşık 10-22 dB, ikincisi yaklaşık 25-26,2 dB) belirtti ve merak edilirse yazara sorulabileceğini söyledi (ekrandan, 01:47:20).

6. Sunum 5: Draco ile ROS'ta 3B Örgü ve Nokta Bulutu Sıkıştırma

Son sunumda bir öğrenci, 2019'da Almanya'da güvenlik ve kurtarma robotları üzerine düzenlenen uluslararası bir sempozyumda yayımlanan, Thomas Wiemann ve arkadaşlarının ROS sensör ve geometri mesajlarının Draco ile sıkıştırılmasına dair çalışmasını anlattı. Sunum özet, giriş, bağlantılı çalışmalar, Draco-ROS uyumu, değerlendirme ve sonuç başlıklarından oluştu.

6.1 Problem ve temel terimler

Sensör teknolojisindeki gelişmelerle ayrıntılı 3B ortam modelleri lazer tarayıcılar ya da RGB-D kameralarla hızlı ve çevrim içi olarak yakalanabilir. Ancak kurtarma ve hizmet senaryolarının çoğunda operatör ile kontrol edilen robot arasındaki bant genişliği sınırlıdır. Bu çalışmada uzaktan kumandalı bir robottan operatörün bilgisayarına ortamın geometrik verisini gönderirken gereken bant genişliğini azaltmak için Google'ın açık kaynaklı geometri sıkıştırma kütüphanesi **Draco**, ROS'a entegre edildi. Çalışmada sıkıştırma oranları, doğruluk ve ortaya çıkan gecikme de analiz edildi.

- **LIDAR (Light Detection and Ranging):** ışıkla nesne tespiti ve mesafe ölçümü yapan teknoloji.
- **RGB-D kamera:** derinlik ölçümü yapabilen kamera.
- **ROS (Robot Operating System):** kabaca bir robot işletim sistemi gibi düşünülebilir. **RViz** ROS'un görselleştirme arayüzüdür.
- **Örgü (mesh):** köşe noktaları (vertex), kenarlar (edge) ve yüzeylerden (face) oluşan yapı.
- **Nokta bulutu (point cloud):** birbirine bağlanmamış noktalardan oluşan veri.

Önerilen yaklaşımda sensörlerden alınan veri gezgin robot üzerinde sıkıştırılır ve ROS'un standart mesajları biçiminde kodlanır. Bu sıkıştırılmış mesajlar robotu kontrol eden uzak masaüstündeki ROS master'a gönderilir. Bir dönüştürme düğümü (kod çözücü) kodlanmış geometri verisini çözer ve RViz'de görselleştirir. Farklı robot uygulama senaryolarını kapsayan referans veri setleri üzerinde kodlama ve kod çözme süreleri ile sıkıştırma oranları ölçüldü.

6.2 Bağlantılı çalışmalar

Önceki çalışmaların ortak noktası, 3B nokta bulutlarından çokgensel yüzey yeniden kurulumunun algoritmik yönlerine odaklanmaktı; bu modellerin verimli sıkıştırılıp gönderilmesine pek eğilinmemişti. Bu çalışma bu boşluğa yönelir.

Geometri sıkıştırmada nokta bulutları için genellikle **octree** (8'li ağaç) tabanlı yöntemler kullanılır. Octree'de bir küp 8 alt küpe bölünür ve alt küplerde veriyle ilgili bilgi tutulur. İletilen veri miktarını azaltmak için ağaç derinliği bant genişliğine göre sınırlandırılabilir. Mevcut yaklaşımlar bağlantı (connectivity) kodlamasını da dikkate alır. Draco örgü verilerini sıkıştırmak için çoğunlukla **Edgebreaker** tekniğine dayanır. Slayta göre Dumanoğlu ve arkadaşları 3B geometri verisinin canlı akışı bağlamında Google Draco, O3DGC, Corto ve OpenCTM'i karşılaştırmış ve iki bulguya ulaşmıştır: Draco yüksek gecikmeli durumlarda diğerlerinden iyi performans gösterir ve ek köşe noktası özniteliklerini etkili biçimde kapsayabilir. Bu çalışmada

malzemeler gibi yüzey özniteliklerinin kodlanmış veri akışına dahil edilmesi için bu kapsamdan yararlanılmıştır (ekrandan, 02:00:00).

6.3 Örgü araçları (Mesh Tools)

ROS'ta 3B nokta bulutları için PCL (Point Cloud Library) formatına dayanan PointCloud2 mesajları vardır, ancak çokgensel veri desteği oldukça kısıtlıdır; basit çokgensel nesneler URDF (Universal Robot Description File) dosyalarıyla desteklenir. Yazarların geliştirdiği örgü araçları yazılımı, örgülerin ROS içine verimli biçimde aktarılmasını sağlar ve 3B geometrileri tanımlamak için standart ROS mesajları tanımlar. Örneğin örgü geometrisi mesajı köşeler, normaller ve yüzeylerle temel geometrik yapıyı; örgü malzemesi mesajı malzemeler ve desen koordinatlarını taşır. Yazılım ayrıca üçgen örgülerin ROS'un TF (transformation) çerçevesinde dönüştürülmesini ve ek köşe ve yüzey özniteliklerinin serileştirilmesini sağlar.

6.4 Draco ile geometri kodlama

Draco, nokta bulutu verileri için k-d ağacıyla noktaları yeniden sıralayan, sıraya göre optimize edilmiş bir kodlama (order-optimized encoding) kullanır. Örgü topolojisini sıkıştırmak için Edgebreaker algoritmasını kullanır. Edgebreaker örgüyü spiral biçimde gezerek kodlar: her üçgen yüzeyin bağlantısını bir string ile kodlar, sonra bu string'ler kütüphane tarafından sıkıştırılarak gönderilir.

Soru (hoca): Resimde orası neden pembe boyanmış ve yeşil çizgi neden yarıda bitiyor? **Cevap:** Sunan öğrenci, görselin makaleden değil başka bir kaynaktan alındığını belirtti. Kodlama örgünün orta noktasından başlar, üçgenlerin kenarlarını izleyerek spiral biçimde bütün örgüyü tarar ve sonucu bir string olarak kodlar. Görselde spiral yalnız bir noktaya kadar gösterilmiştir; gerçekte bütün noktalardan geçerek devam eder.

Draco iç parametrelerin ince ayarına izin verse de sıkıştırmayı esas olarak **niceleme bitlerinin sayısı** belirler. Sunuma göre bu değer en çok 31 olabilir. Bu parametre, hassasiyet ile kodlama ve kod çözme hızları arasındaki dengeyi ayarlamaya olanak verir.

Soru (hoca): Nokta bulutunda elimdeki girdi, şekli oluşturan noktaların koordinat bilgileri mi; sıkıştırmayı bu bilgilere mi uygulayacağım? **Cevap:** Evet. Nokta bulutu, noktaların bağlanmamış halidir ve sıkıştırma bu koordinatlara uygulanır.

Sunumda Draco ile gzip'i karşılaştıran kısa bir video da gösterildi; Draco, gzip işlemini bitirmeden kendi işlemini tamamlıyordu.

6.5 ROS ile bütünleştirme

Slayta göre Draco'nun ROS ile entegrasyonu, mevcut sıkıştırılmamış mesajların değiştirilmesi için önemli bir dönüştürme çabası gerektirmemelidir. Bir örgü geometrisini yayınlamak için iki yol vardır (ekrandan, 02:10:00):

```
Yol 1: Örgü -> ROS -> RViz (doğrudan, sıkıştırmasız)

Yol 2: Örgü -> Örgü Araçları mesajı
       -> Draco kodlayıcı düğümü (sıkıştırma parametrelerine göre Draco veri yapısı)
       -> DracoCompressedData mesajı (bayt akışı)
       -> Draco kod çözücü düğümü
       -> Örgü Araçları mesajı -> ROS -> RViz
```

`DracoCompressedData` mesajı yalnız Draco verisini depolayan bir bayt dizisinden oluşur. Gelecekte sıkıştırma ayarları gibi ek meta veriler eklenebilir. Kod çözücü düğümü, Draco akışını örgü araçları ve PointCloud2 mesajlarıyla uyumlu hale getirir. Her iki düğüm de istenen sıkıştırma oranı ve doğruluğuyla parametrelendirilebilir.

6.6 Değerlendirme

Yapı, iki farklı tarayıcıyla (lazer tarayıcı ve LIDAR) elde edilmiş üçgen örgüler ve nokta bulutları üzerinde test edildi. Yapılandırılmış iç ortamların yanında karmaşık dış ortamlardan da referans veri setleri seçildi; bunlardan biri bir üniversitenin botanik bahçesidir. Botanik

bahçesi örgüsü 25, 15 ve 10 niceleme bitiyle sıkıştırılıp yeniden kuruldu. Ortadaki bir LIDAR sensörü 360 derece lazer atımları gönderir; yansımanın geldiği uzaklığa bir nokta koyarak çevresinin nokta bulutunu oluşturur.

Değerlendirmede farklı sıkıştırma seviyelerinde sıkıştırma oranları, kodlama ve kod çözme süreleri ve geometrik hassasiyet incelendi. En yüksek veri azaltımı için en yüksek sıkıştırma ön ayarı kullanıldı. Veri boyutunu ve geometrik hassasiyeti etkileyen ana öğe niceleme bitleridir. Niceleme biti azaldıkça ayrıklaştırma hataları artar ve ortamın daha az gerçekçi bir temsili elde edilir; 8 bitlik bir örnekte bu bozulma açıkça görülür.

Geometrik hatayı ölçmek için orijinal sıkıştırılmamış veri ile kodu çözülmüş mesajlar arasındaki metrik mesafeler, açık kaynaklı CloudCompare yazılımıyla analiz edildi (ekrandan, 02:16:30). 25 bitlik ayarda yüksek sıkıştırma elde edilirken örgüler ve nokta bulutları için geometrik hata ihmal edilebilir düzeydedir. Ofis veri setindeki hatalar Bremen veri setindekilerden düşüktür. 15 bite kadar mesafe hatası kabul edilebilir düzeyde kalır; daha az bitte hatalar artar.

Kodlama ve kod çözme süreleri şöyledir (ekrandan, 02:18:45):

Kuantizasyon	Bina enc.	Bina dec.	Botanik enc.	Botanik dec.	Ofis enc.	Ofis dec.	Bremen enc.	Bremen dec.	Velodyne enc.	Velodyne dec.
Q5	9972,00	2367,00	24091,00	7076,00	719,84	706,78	1828,70	2043,34	3,41	3,97
Q10	9980,00	2419,00	23931,00	7082,00	1239,27	912,44	2470,75	2176,43	6,88	5,37
Q15	9964,00	2643,00	24147,00	7854,00	1439,83	1060,80	3813,87	2953,39	7,96	5,95
Q20	10210,00	2567,00	24649,00	7445,00	1463,05	1126,32	3964,70	3153,21	7,92	6,33
Q25	10427,00	3107,00	25106,00	7762,00	1521,66	1188,27	4171,64	3335,37	8,12	6,62

Bütün süreler milisaniyedir. Kodlama süreleri niceleme seviyesi arttıkça genel olarak artar. Örgülerde kod çözme kodlamadan belirgin biçimde hızlıdır (örneğin Bina'da yaklaşık 10.000 ms'ye karşı 2.400 ms civarı). Nokta bulutlarında iki süre birbirine yakındır (Ofis Q5'te 719,84 ms ve 706,78 ms). ROS'un da zaman kısıtları vardır ve Draco parametrik olduğu için bu kısıtlara göre ayarlanabilir.

> **Ders sonrası not:** Sunumda Velodyne verisi için kodlama sürelerinin 3 ila 8 saniye olduğu söylendi; tabloya göre bu süreler 3,41-8,12 **milisaniyedir**. Velodyne LIDAR 20 Hz'e kadar tarama yapar, yani bir tarama yaklaşık $1000/20 = 50$ ms sürer. Kodlama süresi bu sürenin altında kaldığı için veri tarandığı hızda, gerçek zamanlı olarak kodlanabilir.

6.7 Sonuç ve tartışma

Çalışma, ROS ile Draco arasında örgü ve nokta bulutlarını sıkıştırmak için bir yapı sunar ve bunu sıkıştırma oranı, harcanan zaman, veri akışlarının kodlanması ve geometrik hassasiyet açısından değerlendirir. Sonuçlara göre araçlar, sensörlerin kayıt hızına göre gönderilen veri miktarını gerçek zamanlı olarak azaltabilir. Ortam yapısı hassasiyeti etkiler. 25 niceleme bitiyle yapılan sıkıştırma iletilen veriyi önemli ölçüde azaltır ve veri yarı kayıpsız biçimde iletilir: yüzde yüz kayıpsız değildir, ama kayıplar kabul edilebilir düzeydedir. Gelecekte üretilen veri akışlarının paket kayıplarına ve yüksek gecikmelere karşı ne kadar sağlam olduğu incelenecektir.

Soru (bir öğrenci): Ortamda hareketli nesneler olduğunda nokta bulutu ya da örgü değişmez mi? **Cevap:** Değişir. Sensör her taramada nesnenin güncel konumuna göre nokta koyar. Örneğin sürücüsüz bir otomobilin üstündeki LIDAR yanından geçen bir arabayı nokta bulutu olarak tanımlar; araba geçip gidince o nokta bulutu kaybolur. Bunları temizlemek için sıkıştırmadan önce çeşitli algoritmalar eklenir.

Hoca bu durumu videoya benzetti: videolarda da kareler arasındaki farklar kodlanır. Bir öğrenci de istatistiksel yöntemlerdeki gibi, hareketsiz bir nesne tanımlandıktan sonra daha küçük bir bit sayısıyla, hareketli nesnelerin ise farklı bit sayılarıyla kodlanabileceğini; bunun harflerin frekanslarına göre kod verilmesine benzediğini ve yeni sıkıştırma algoritmalarına yol açabileceğini söyledi. Hoca da 3B ortam modellerinin sıkıştırılıp aktarılmasının yeni gelişen bir alan olduğunu belirtti.

Örnek Sorular

1. (derste sözlü, 00:24:24) Bir deprem kaydında önce hafif salınımlar, sonra büyük salınımlar, sonra yeniden sönümlenme vardır. Bu işaret fark (residual) alınarak sıkıştırılacaksa veri tek blok olarak mı, yoksa küçük bloklara bölünerek mi sıkıştırılmalıdır?

> **Cevap (derste verildi):** Küçük bloklara bölünerek. Büyük salınım olan kısımlarda farkların varyansı büyür; bütün veri tek blokta sıkıştırılırsa varyans artar, entropi yükselir ve sıkıştırma kötüleşir. Sunumdaki çalışmada da veri $N = 5, 10, 15$ segment değerleriyle parçalanmıştır.

2. (derste sözlü, 00:42:47) Bir yöntemin sıkıştırma oranı 0,36 olarak verilmiştir. Sıkıştırma performansı (kazanç) yüzde kaçtır?

> **Cevap (derste verildi):** $1 - 0{,}36 = 0{,}64$, yani %64.

3. (derste sözlü, 00:43:08) Unigram, bigram ve trigramların aynı Huffman yapısına konduğu kelime tabanlı bir sıkıştırmada ağaç boyutu hakkında ne söylenebilir? Ağacın derinliğini kestirmek için hangi bilgiye ihtiyaç vardır?

> **Cevap (derste verildi):** Hepsi aynı ağaçtaysa ağacın seviyesi yüksek olur. Derinliği kestirmek için entropi değeri gerekir; entropiden ortalama kaç bit kullanıldığı ve dolayısıyla ağacın yaklaşık derinliği çıkarılabilir. Belge paragraf paragraf işleniyor ya da bağlam değiştikçe ağaç yeniden kuruluyorsa ağaç o kadar derin olmaz.

4. (derste sözlü, 00:37:11) Kayan bağlam penceresi kullanan bir sıkıştırıcı konunun değiştiğini nasıl anlar?

> **Cevap (derste verildi):** Pencereye gelen yeni kelime pencere içindeki hiçbir kelimeyle örtüşmüyorsa bu kelime daha önce geçmemiş demektir; buradan bağlamın (konunun) değiştiği sonucu çıkarılır ve o konuya ait, önceden hazırlanmış sözlüğe ve Huffman ağacına geçilir.

5. (derste sözlü, 01:07:10) Metin sınıflandırma ve dil modelleme veri kümeleri üzerinde çalışan Bayesçi bir yöntem kayıplı mı, kayıpsız mı sıkıştırma yapar?

> **Cevap (derste verildi):** Metnin kendisine kayıplı sıkıştırma uygulanamaz. Bu çalışmada sıkıştırılan metin değil, RNN modelinin parametreleridir; modelin ağırlıklarının çoğu sıfırlanarak daha az parametreyle benzer başarı elde edilir.

6. (derste sözlü, 01:40:17) Bir tabloda sıkıştırma oranı "1:100", başka bir tabloda yalnız "100" olarak verilmiştir. Bu ne anlama gelir?

> **Cevap (derste verildi):** İkisi de aynıdır: 100 bit 1 bite sıkıştırılmıştır. Benzer şekilde 1:5, 5 bitin 1 bite sıkıştırılmasıdır.

7. (ekrandan, 01:40:20 ve derste sözlü, 01:41:32) Model 1'in tablosunda sıkıştırma oranı 2:1'den 100:1'e çıkarken NRMSE 0,005'ten 0,048'e yükselmekte, SNR 40,666 dB'den 20,295 dB'ye düşmektedir. RMS hata ile SNR arasındaki ilişki nedir ve tablo bunu nasıl gösterir?

> **Cevap (derste verildi):** RMS hata ile SNR ters orantılıdır. RMS hata orijinal veri ile sıkıştırılıp açılmış veri arasındaki farktır; küçük olduğunda veri orijinale yakındır ve SNR büyüktür. Tabloda sıkıştırma oranı arttıkça bozulma artar: NRMSE büyür, SNR küçülür.

8. (derste sözlü, 02:05:33) Bir nokta bulutu sıkıştırılırken girdi olarak ne kullanılır?

> **Cevap (derste verildi):** Noktaların koordinat bilgileri. Nokta bulutu, noktaların birbirine bağlanmamış halidir ve sıkıştırma bu koordinatlara uygulanır.

9. (ekrandan, 02:18:45) Velodyne LIDAR 20 Hz tarama frekansıyla çalışmakta, Draco ile kodlama süresi en çok 8,12 ms olmaktadır. Sıkıştırma gerçek zamanlı yapılabilir mi?

> **Çözüm (derste verilmedi):** 20 Hz tarama, bir taramanın $1000/20 = 50$ ms sürmesi demektir. En uzun kodlama süresi 8,12 ms < 50 ms olduğu için her tarama bir sonraki tarama gelmeden kodlanabilir; sıkıştırma gerçek zamanlı yapılabilir.

Dersin Toparlaması

Bu derste seminer haftaları başladı ve beş makale sunuldu. İki sunum sismik verinin sıkıştırılmasıydı: birincisi DNN ile tahmin yapıp fark değerlerini nicemleyerek Huffman ile kodlayan, LPC'den daha düşük varyanslı farklar ve daha yüksek SNR elde eden bir yöntem; ikincisi evrişimli otokodlayıcılarla doğrudan sıkıştırma yapan ve sonuçlarını SNR, NMSE ve NRMSE ile ölçen bir yöntemdi. İki sunum doğal dil işlemeyle ilgiliydi: biri dil modelinin bigram bağlamına göre ayrı Huffman ağaçları kurarak kelime tabanlı metin sıkıştırma yapan bir yöntem, diğeri RNN'lerin parametrelerini Bayesçi seyreltmeyle binlerce kat azaltan bir model sıkıştırma yöntemiydi. Son sunum, Draco kütüphanesiyle robotlardan gelen 3B örgü ve nokta bulutlarının ROS üzerinde sıkıştırılmasını anlattı.

Hoca bütün sunumları dönem boyunca görülen kavramlara bağladı: fark alarak sıkıştırma ve bloklara bölmenin önemi, varyans ile entropi ilişkisi, sıkıştırma oranı ile sıkıştırma performansının farkı, Huffman ağacının derinliği ile entropi ilişkisi, LZ'deki kayan pencere fikri ve RMS hata ile SNR'nin ters orantısı. Seminer sunumları sonraki derslerde de devam edecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.